A stabilizer-rank backend for `clifft`: one compiler, two executors

Research note

July 12, 2026

Abstract

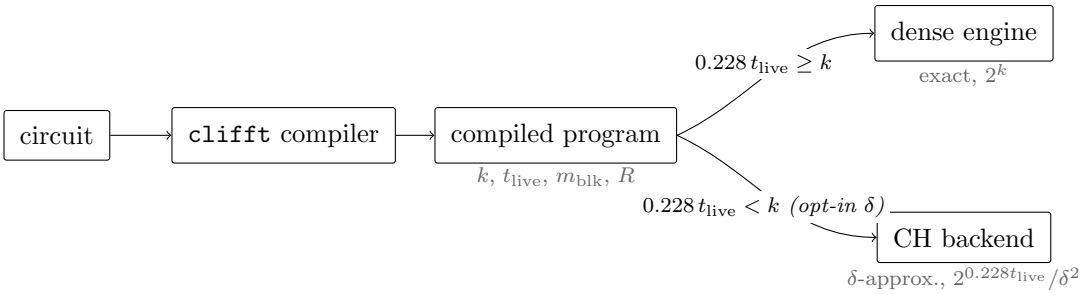
`clifft` simulates Clifford+T circuits exactly, with a symbolic Clifford frame and a dense 2^k active block whose size k (the compiled *peak rank*) is the cost driver. We add a second executor: a low-rank stabilizer (CH-form) backend whose cost driver is instead the circuit's live magic — $2^{0.228\,t}$ terms for t live T-gates, at a tunable accuracy δ. Both cost exponents are known from a single `clifft` compilation before anything runs, so execution becomes a per-program dispatch: the dense engine where the compiled block is small (most circuits), the CH backend where the block is large but magic is sparse. Measured against `clifft`'s exact fast path on circuits that compile to genuinely large blocks, the CH backend crosses over at $n \approx 34$ (coarse accuracy) and $n \approx 48$ (8% error), and past $n \approx 62$ — where every known exact method needs 16 GB–1 TB — it is the only method that runs (n=72 in 267 s and 1.9 GB). Accuracy follows a clean measured law, TV $= (0.50 \pm 0.01)\,\delta$. The CH backend consumes `clifft`'s optimized Heisenberg IR directly (which contains no Clifford gates at all — the compiler absorbs them), handles arbitrary interleaved magic by T-gadgetization, adaptive feedforward circuits online, and episode-structured circuits at a per-episode cost exponent (m_{blk} instead of t), all validated to machine precision against `clifft` and an independent exact evaluator.

1 The system in one view

`clifft`'s state is split into a *symbolic Clifford frame* (dormant axes evolve for free) and a *dense active block*, a 2^k statevector expanded only for non-Clifford content. Two static facts scope the design (`research/stabrank_profiling`): k is built from magic — plain Clifford superposition never inflates it — and `clifft`'s measurement-reordering passes shrink k aggressively. When the block is nevertheless large but *magic-sparse*, its stabilizer rank is $\ll 2^k$; the CH backend executes exactly that regime, at the cost of exactness.

One compiler. Every circuit goes through `clifft`'s compiler regardless of which engine will execute it. Tracing produces the Heisenberg IR: a remarkable object that contains *no Clifford gates at all* — every Clifford is absorbed into the Pauli strings of the surviving ops, so what remains is a sequence of T-rotations about arbitrary Paulis, projective Pauli measurements, and classically-controlled Paulis (feedforward). The optimization passes then merge magic ($T \cdot T \to S$ and similar), leaving the *live* T-count t_{live}, and reorder measurements as early as they commute — which is what keeps the compiled peak rank k small, usually far smaller than the qubit count.

Two executors, dispatched at compile time. The compiled artifact carries every quantity the routing decision needs:



The dispatcher (`dispatch.py`) prices four strategies from one compilation and routes each program to the cheapest feasible one:

strategy	cost ($\log_2$ dominant term)	guarantee	inputs
dense engine	k	exact	k = peak_rank
CH backend, global	$0.228\,t_{\text{live}} + 2\log_2\frac{1}{\delta}$	δ-approx.	t_{live} (HIR)
CH backend, episodic-exact	m_{blk}	exact	m_{blk}, R (profiler)
CH backend, episodic-approx.	$0.228\,m_{\text{blk}} + 2\log_2\frac{R}{\delta}$	δ-approx.†	m_{blk}, R

m_{blk} = peak T-count within a single *episode* (one grow-and-fully-collapse cycle of the active block); R = episode count. †measured caveat for full-record targets, §5.6.

Representative routings (`dispatch.py` demo, δ=0.15): dense IQP (*instantaneous quantum polynomial-time*: commuting T+CZ circuits between Hadamard layers) at n=40 (k=20, t=40) → CH backend, global; random Clifford+T and hidden shift (which compile to $k \leq 10$) → dense; a feedforward conveyor with 12 rounds of 128 T's (k=49: dense infeasible; $0.228 \cdot 1536 \approx 350$: global absurd) → CH backend, episodic — the only strategy that runs.

What comes from where. The CH backend borrows `clifft`'s two architectural ideas — the global symbolic Clifford frame and measurement-driven collapse of the non-Clifford core — and re-instantiates them over a different core: BGH's low-rank CH-form sum [1] instead of a dense 2^k array. The simulation mathematics (CH-form, minimal-extent decomposition, sparsification, norm estimation, gadgetization) is BGH/Bravyi–Gosset throughout; none of `clifft`'s runtime is used. `clifft` serves as compiler front-end (the CH backend consumes its optimized HIR directly, §4.1), as validation oracle (every component is checked against it to machine precision), and as the exact opponent in every benchmark.

Design commitments. (i) The CH backend is an explicit opt-in (δ), never a silent substitute: when exactness is demanded past the memory wall, the honest answer is "infeasible" — `clifft`'s exactness contract is untouched. (ii) Both executors consume the same compiled stream, so every compiler improvement (magic merging, measurement reordering, fusion) lowers both engines' costs. (iii) Baseline discipline: all comparisons below run against `clifft`'s true fast path — `record_probabilities` on the *measured* program — and are cross-checked by an independent exact evaluator; comparing against the unitary path (peak_rank = n) would overstate the CH backend by orders of magnitude and is never done here.

In one sentence: `clifft`'s architecture is the blueprint, BGH's mathematics is the material, and `clifft` itself is the compiler front-end that decides — per program, before execution — which executor runs.

2 Background and related work

CH-form and low-rank decompositions. A stabilizer state is stored as $|\phi\rangle = \omega\, U_C U_H |s\rangle$ (Bravyi–Browne–Calpin–Campbell–Gosset–Howard [1]): $O(n^2)$ bits per term, $O(n)$ Clifford gates, $O(n^2)$ Hadamard and single-amplitude queries. A magic state is a sum $|\psi\rangle = \sum_a c_a |\phi_a\rangle$ of χ terms; Cliffords preserve χ, each T doubles it, measurement can collapse it.

Related work. "Cliffords paid once, exponential cost only on the magic" is the architecture of Bravyi–Gosset [2]; Qassim, Wallman and Gosset [3] recompile circuits into non-Clifford rotations times a single global Clifford precisely to speed up CH-form simulators; García–Markov [6] shared a stabilizer frame across superposition terms; Pauli-based computation [7] absorbs all Cliffords into adaptive Pauli measurements on t magic qubits. The closest single work is Pashayan et al. [5]: gadgetize the T's, then *statically* compress the global Clifford and the outcome projector before running a stabilizer-rank estimator. Stabilizer tensor networks with magic injection [8] pair a Clifford frame with an MPS residual; `quizx` [9] reaches "decompose only the magic" via ZX rewriting. What this system adds is the *online* form of the composition — a compiler-maintained global frame over a CH-form term sum, with dynamic measurement-driven collapse and mid-circuit/adaptive measurement — plus the engineering observation (verified in qiskit-aer's source) that the flagship open-source CH-form simulator, `extended_stabilizer`, applies every Clifford per-term at $O(\chi)$ cost, with no frame. The composition's advantage is a constant/poly factor, never an exponent, and we claim nothing more for it.

3 Theory

3.1 Minimal-extent magic decomposition (the 0.228 exponent)

$T = \alpha I + \beta S$ with $\alpha = \frac{1}{2} + i\frac{\sqrt{2}-1}{2}$, $\beta = \bar{\alpha}$, both terms Clifford, $|\alpha| + |\beta| = 2^{0.114}$, so the stabilizer extent per T is $2^{0.228}$ — optimal within the extent framework, since extent is multiplicative for tensor products of single-qubit states [1]. Branching in the computational basis instead costs extent 2^t, which sparsification cannot repair; this basis choice is the crux.

3.2 Single-shot sampling, and gadgetization for general circuits

Importance-sampling k terms from $p_a = |c_a|/\|c\|_1$ gives an unbiased $|\omega\rangle$ with $\mathbb{E}\||\psi\rangle - |\omega\rangle\|^2 = (\|c\|_1^2 - \|\psi\|^2)/k$; $k \sim 2^{0.228t}/\delta^2$ bounds the 2-norm error by δ. For *product* magic (one T-layer, as in IQP) the branch strings factorize and are sampled directly — unbiased, no 2^t built, no factor-t variance accumulation (streaming mid-circuit sparsification pays that factor; it is implemented but not the benchmarked path).

Arbitrary circuits with interleaved magic reduce to this case by **T-gadgetization**: teleport each in-line T onto its own ancilla and force the gadget outcome to 0. Since the gadget's classically-controlled correction makes all outcome branches equivalent, the 0-branch alone carries the answer — each forced-0 gadget contributes exactly $T/\sqrt{2}$, so $P(x) = 2^t\, |\langle x, 0^t | C_{\text{gadget}} (|0^n\rangle \otimes |T\rangle^{\otimes t})|^2$ with C_{gadget} entirely Clifford. All magic is again one product layer: the single-shot sampler applies with no streaming penalty, and the normalization stays analytic (§3.4).

3.3 Episodic execution

Long computations *recycle* magic: the active core grows and fully collapses again (an *episode*), so the per-episode magic m_{blk} stays bounded while t_{live} grows with runtime. The CH backend exploits this by running one episode at a time. The sound schedule, each element load-bearing (measured in §5.6):

- **Sparsify at T-time only.** The resample is well-conditioned only when $\|c\|_1$ is extent-controlled, which holds for the T-branching of a freshly collapsed base. It is *never* applied after a projection: forced measurements inflate $\|c\|_1/\|\psi\|$ (cancellation) and make the resample ill-conditioned.
- **Collapse exactly at boundaries.** When an episode has fully collapsed the state is rank 1; the collapse is a deterministic amplitude-ratio sum at an analytic tableau support point ($x^* = Gs$ over $\mathbb{F}_2$, using $F^{-1} = G^\top$), detected by a two-probe parallelism test (`collapse_if_parallel`) — $O(\chi n^2)$, no 2^n scan.
- **Debias with two independent runs.** The naive estimator $\|\Pi\omega\|^2$ is biased upward by the sparsification variance; the cross estimator $\langle \Pi\omega | \Pi\omega' \rangle$ of two independent runs is unbiased, and at rank-1 endings reduces to two amplitudes at one support point.
- **Keep episodes contiguous.** Episodic mode uses the compiler's magic-merging pass only (`PeepholeFusionPass`); the measurement-hoisting pass would interleave episodes and defeat the boundary collapse.

At budget $2^{m_{\text{blk}}}$ (episodic-exact) no resample ever fires and the run is exact at cost exponent m_{blk}, deterministically. Below that budget the run is unbiased but approximate, with errors compounding over the R boundaries — the compounding law is measured in §5.6.

3.4 Normalization

$P(x) = |\langle x|\psi\rangle|^2/\|\psi\|^2$ needs $\|\psi\|^2$ without materializing 2^n. Three regimes, in order of preference: *analytic* — for unitary product magic (including everything gadgetized) $\|\psi\| = 1$ and $\mathbb{E}\|\omega\|^2 = 1 + (\|c\|_1^2 - 1)/k$ exactly, so no estimation enters; *rank-1* — at fully-collapsed endings the norm is an $O(\chi n^2)$ amplitude-ratio sum; *estimated* — the BGH estimator (Lemmas 2–4, $O(n^3)$ exponential sum) otherwise. The estimator's per-sample relative spread is ≈ 1.2, so L samples cost $\approx 1.2/\sqrt{L}$ relative error on every reported probability; keeping that subdominant for generic circuits scales as $\chi L \sim 2^{0.228t}/\delta^4$ — the pipeline's one real accuracy-cost bottleneck, stated as such. Quoted accuracies always include the normalization term.

3.5 Error guarantee, stated precisely

Sparsification bounds the *2-norm* error $\|\psi - \omega\| \le \delta$. For a single target that yields an *additive* amplitude error of typical size $\delta\, 2^{-n/2}$ — i.e. a $\sim \delta$ *relative* error on *typical* (anticoncentrated) probabilities $P(x) \sim 2^{-n}$, which is what the measured TV $\approx 0.5\,\delta$ law reflects. It is **not** a per-x relative-error guarantee: individual small probabilities can be off by more. Guaranteed relative error costs the exact-rank methods ($2^{0.396t}$ [4]); multiplicative approximation remains GapP-hard for T-type IQP [10], so there is no poly-time escape.

3.6 The online frame, for adaptive circuits

For circuits the compiler can see through, the Heisenberg IR already delivers "free Cliffords" statically. For *dynamic* circuits — mid-circuit measurement with data-dependent control the compiler cannot resolve — the CH backend maintains the frame online: $|\text{state}\rangle = F(\sum_a |\phi_a\rangle)$ with F a global Clifford updated in $O(n)$ per gate; magic and measurement act on the terms conjugated through F at the optimal $\{I, S\}$ extent. The frame does not eliminate per-term work; it *moves* it from Cliffords onto magic and measurement (conjugated operators $P' = F^{-1}PF$ have weight $\sim w$, costing $O(w)$ per-term applications per T and per measurement). That is why the frame pays off only on Clifford-dominated adaptive circuits — the measured phase boundary of §5.5.

4 Implementation and validation

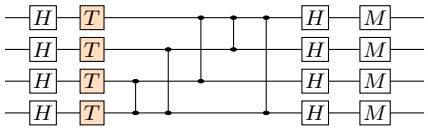
A Python engine (`research/chform_backend`) is the reference implementation — including the HIR bridge, gadgetization, episodic execution, the online frame, and the dispatcher — and a standalone bit-packed C++20 port (`research/chform_cpp`) is the performance implementation for the build-and-query pipeline (single-shot magic + Clifford stream + amplitude/norm queries; amplitudes are word-based, and the multi-word $n > 64$ paths are covered by tests). Ground truth everywhere is `clifft`'s exact `record_probabilities`, independently cross-validated by a meet-in-the-middle (MitM) evaluator (`mitm_iqp.cpp`): exact $P(x)$ for any T+CZ IQP circuit in $O(n2^{n/2})$ time and $2^{n/2}$ memory — ground truth to $n \approx 62$ on a 36 GB machine, and incidentally the fastest exact method on that family.

Table 1: Validation (max abs. error unless noted).

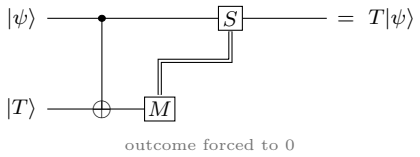
Component	Reference	Error
CH-form amplitude (Eq. 56)	hand-built states	3×10^{-17}
Full Clifford set	dense, 2000 circuits	8×10^{-15}
Single-qubit projection	dense, 2000 states	5×10^{-15}
Whole engine vs. `clifft`	`get_statevector`	2×10^{-16}
W=2 (n=80) embedded circuits	dense oracle, 900 runs	6×10^{-15}
Norm est. Lemma 4 (exp. sum)	brute force	0 (exact)
Norm est. Lemma 3	dense	5×10^{-16}
Norm est. Lemma 2	exact, L=6000	3.0%
W=2 (n=68) norm est., unit-norm state	exact (=1), L=600	4.8%
Frame engine vs. plain (incl. measurement)	forced outcomes	6×10^{-15}
Gadgetized = dense = `clifft` ($7T$ CCZ; random)	interleaved circuits	10^{-14}
HIR bridge (400 fuzzed circuits + families)	`record_probabilities`	2×10^{-13}
Episodic-exact conveyor (feedforward replayed)	`record_probabilities`	4×10^{-17}
Analytic support point	500 random tableaux	nonzero amplitude
MitM IQP evaluator	dense / `record_probabilities`	6×10^{-16} / 6×10^{-14}

4.1 Consuming the compiler's output

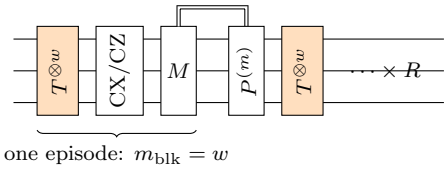
The CH backend reads `clifft`'s optimized HIR directly (`hir_bridge.py`, via the exposed `parse→trace→HirPass` pipeline). The IR's op types map one-to-one onto the engine's explicit-Pauli primitives: T-rotations about arbitrary Paulis (`rz_about_pauli`), projective Pauli measurements (`measure_pauli_forced_fast`),



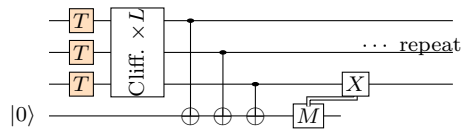
(a) **Dense random IQP** (n=4 sketch): T on every qubit, CZ on each pair w.p. $\frac{1}{2}$. $t = n$; the measured program compiles to peak rank $n/2$. Carries the crossover, dial, frontier, and external baselines (§5.1–5.3, §5.7).



(b) **The T-gadget** behind gadgetization: an in-line T becomes a CX onto a $|T\rangle$ ancilla, a measurement, and a conditional S. Turns any interleaved circuit into one product magic layer; exercised at scale by hidden shift (up to 96 qubits) and random Clifford+T (§5.4).



(c) **Feedforward magic conveyor**: each round injects w T's, mixes, and measures the block away with conditional-Pauli feedforward. Per-episode magic $m_{\text{blk}} = w$ stays bounded while $t_{\text{live}} = wR$ grows. Carries the episodic measurements (§5.6) and the dispatch demo (§1).



(d) **Cultivation-style patch**: data accumulates injected T's; each round runs syndrome extraction (CX fan, ancilla measurement, conditional correction) plus L layers of code-preserving logical Clifford traffic. Carries the adaptive frame benchmark (§5.5); teleported-T injection is panel (b) run as a live adaptive gadget.

Figure 1: The circuit families behind the benchmarks (schematics; benchmark instances scale the same structure to $n = 16$–96).

and conditional Paulis (feedforward, deterministic under a forced record). Consequences, measured: the CH backend performs *zero* Clifford gate applications on any compiled circuit; χ tracks $2^{t_{\text{live}}}$ and often less (the compiler's hoisted measurements collapse terms mid-run — $\chi_{\text{peak}} = 8$ where $2^{t_{\text{live}}} = 64$ on one family); and t_{live} is roughly half of the raw T-count on random circuits (32/40/48 → 18/28/28), which is why the dispatch rule uses it. For compile-known circuits the compiler thus supersedes the online frame of §3.6, which remains the mechanism for dynamic circuits. The C++ scale arm currently consumes plain gate streams; feeding it HIR needs a simultaneous reduction of the terminal Pauli measurements (a symplectic elimination) — future work, so its benchmark numbers below are conservative for the CH backend.

5 Benchmarks

Contestants run single-threaded on an Apple M-series workstation (36 GB; `clifft` on its scalar/NEON path); the MitM ground truth uses all cores but competes with no one. Scripts and raw JSON are committed. Measured component facts: the L_1 growth per T is exactly $2^{0.114}$; sparsification error tracks the BGH bound to $\sim 1\%$; the single-shot estimator is unbiased. Figure 1 sketches the four circuit families that carry the measurements.

5.1 The crossover

Head-to-head on circuits that compile to *genuinely large* blocks: dense random IQP (CZ on each pair w.p. $\frac{1}{2}$; the measured program compiles to peak_rank $= n/2$, so the exact baseline scales as

$2^{n/2}$ — as does MitM; everything exact costs $2^{n/2}$ on this family). 96 uniform targets, ground truth by MitM cross-checked against `record_probabilities`; backend at budget $k = 2^{0.228n}/\delta^2$, analytic normalization, timing includes build + normalization + all amplitude evaluations; the error metric charges the full pipeline, $\mathrm{TV} = \frac{1}{2}\sum_x |\hat{P}(x) - P(x)| / \sum_x P(x)$. Means over 3 realizations ($n \le 48$; 1 above):

Table 2: Exact fast path vs approximate backend (seconds for 96 $P(x)$).

n	k_{meas}	`clifft` exact	ours δ=0.4	TV	ours δ=0.15	TV
24	12	**0.003**	0.09	0.070	0.08	0.070
32	16	**0.06**	0.15	0.140	0.50	0.073
36	18	0.27	**0.18**	0.192	1.22	0.081
40	20	0.77	**0.41**	0.175	2.94	0.079
44	22	4.1	**1.0**	0.194	7.4	0.076
48	24	17.3	**2.4**	0.197	**16.9**	0.082
52	26	92.9	**5.5**	0.186	**39.4**	0.074
56	28	398	**12.1**	0.165	**86.2**	0.078
60	30	≥16 GB, skipped	**27.7**	0.185	**196**	0.086

The crossover sits at $n \approx 34$ for TV ≈ 0.17 and $n \approx 48$ for TV ≈ 0.08; `clifft`'s exact path grows as $2^{n/2}$ on the nose ($\times 4.6$ per $+4$ qubits) and exits at $n \approx 58$–62 on memory, while the CH backend grows as $2^{0.228n}$ ($\times 1.9$ per $+4$). The axis is the *compiled peak rank*, not the qubit count: on families whose measured programs compile small (nearest-neighbour-CZ chains: peak_rank $= 1$; generic random circuits: ≤ 6), `clifft` answers in microseconds and the CH backend has no role — the dispatch rule of §1 encodes exactly this.

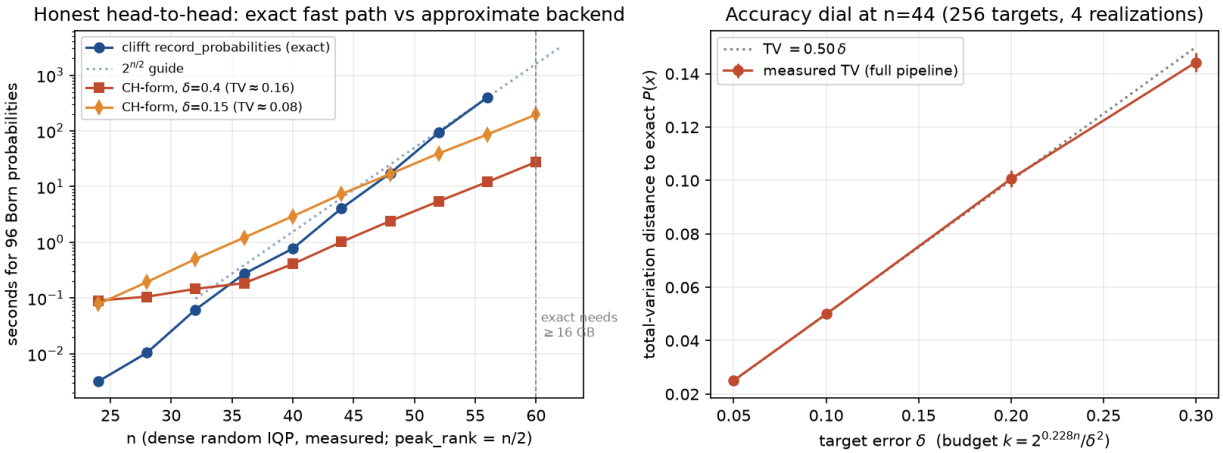


Figure 2: *Left:* seconds for 96 Born probabilities on measured dense IQP: `clifft`'s exact forced-replay path vs the CH backend at two accuracy settings; the dotted guide is $2^{n/2}$. *Right:* the accuracy dial at n=44: full-pipeline TV against exact $P(x)$ vs the target δ, with the TV $= 0.50\,\delta$ line.

5.2 The accuracy dial

At n=44, 256 targets, 4 realizations, analytic normalization (Fig. 2, right): TV = 0.144(4), 0.101(3), 0.0500(6), 0.02 at $\delta = 0.30, 0.20, 0.10, 0.05$ — i.e. TV = $(0.50 \pm 0.01)\,\delta$, with runtime $\propto k \propto 1/\delta^2$ (2.0 → 73.9 s). The law holds for the full pipeline where normalization is analytic; with an L=30 estimated norm the same budgets degrade by up to $\sim 2\times$ on unlucky draws (0.07–0.32 observed vs 0.07–0.21 analytic), exactly the $1.2/\sqrt{30} \approx 22\%$ multiplicative noise of §3.4 — the normalization term belongs in any quoted accuracy.

5.3 The frontier

Dense IQP, δ=0.4, single-shot build + one amplitude (`bench_iqp.cpp`); the exact column is what any known exact method pays on this family.

Table 3: Past-exact frontier (dense IQP, δ=0.4).

n	χ	build	CH-form mem	exact $2^{n/2}$ mem
40	3 477	0.4 s	3.6 MB	16 MB (feasible)
50	16 889	3.0 s	22 MB	512 MB (feasible)
60	82 029	25 s	128 MB	16 GB (marginal)
66	211 728	82 s	683 MB	137 GB (infeasible)
72	546 497	267 s	1.9 GB	1.1 TB (infeasible)

Exact ground truth exists to $n \approx 62$ (MitM), where the measured TV still follows the dial (0.086 at n=60, δ=0.15). At n=66–72 no exact check exists (that is the point); accuracy there rests on the δ-law holding at every checkable size, the analytic normalization being exact for this family, and the multi-word code paths being tested (Table 1) — an extrapolation, stated as such.

5.4 Generality: gadgetized circuits

The gadgetized route (§3.2) handles arbitrary interleaved magic at the product-magic price (`gadgetize.py`; C++ arm `bench_gadget.cpp`); the norm-estimation bottleneck of §3.4 never enters, since normalization stays analytic. At scale, the hidden-shift family (bent-function construction, CCZ oracles, 14 T's per CCZ) provides *built-in exact ground truth*: the output is the shift s deterministically, so $P(s) = 1$ at any size. Measured at $\delta = 0.3$: n=16/24/32/40 (t = 28–56, up to 96 total qubits) give $\hat{P}(s) = 0.99/0.94/0.90/0.92$ in 0.1–8 s, with $\hat{P}(x \neq s) = 0$. On random interleaved Clifford+T vs `clifft` exact (targets sampled from the output distribution — the likelihood use-case): TV 0.04–0.09 at $\delta = 0.15$. These families validate generality rather than exhibit a win regime: `clifft` compiles both to peak_rank ≤ 10 and answers exactly in microseconds — which is precisely the dispatch rule at work.

5.5 Adaptive workloads and the phase boundary

For dynamic circuits the online frame (§3.6) is measured against a plain per-term engine (the architecture of Aer's `extended_stabilizer`, verified in its source) on identical trajectories — same circuits, same forced outcomes, same term store and measurement code; only who pays for Cliffords differs. All engine variants agree to 3×10^{-16}. On a synthetic sweep (a width-w register, rounds of feedforward + $T^{\pm}$ layer + D global Clifford scrambling layers applied *while* $\chi = 2^w$): frame time is flat in D (6.9–7.9 s from D=4 to 96 at w=8) while per-term time grows linearly (3.1 → 65.9 s); the

crossover sits at $D^* \approx 1.2w$, with 8.4× at D=96 and growing (Fig. 3, right). Below the boundary the frame costs up to ∼2× — the conjugation price of §3.6: at w=12, D=4 the plain engine spends 68 s on per-term Cliffords where the frame spends 3 ms, yet the frame loses overall because the moved work lands on magic and measurement.

Natural protocols land on the winning side. Two recognizable families locate real protocols relative to the boundary (`bench_natural.py`; identical-trajectory replay as above): *cultivation-style patches* — a data register accumulates injected T's while every round runs syndrome extraction (ancilla CX fans, measurements, classically-controlled corrections, ancilla reuse) plus L layers of code-preserving logical Clifford traffic — and *teleported-T injection*, the textbook adaptive magic gadget. Measured (8 data qubits, 7 checks, up to 10 injected T's, χ up to $\sim 10^3$):

Table 4: Natural adaptive workloads: plain (per-term Cliffords) vs frame, identical trajectories.

workload	Cliffords/round	frame speedup
bare syndrome extraction (L=0)	∼14	0.8–1.0× (at the boundary)
+ 1 logical-traffic layer	∼32	3.7–6.7×
+ 4 layers	∼68	7–16×
teleported-T gadget	∼4/gadget	1.7–2.8×

Bare syndrome extraction sits exactly at the boundary; any logical Clifford traffic pushes the composition into clear wins — the plain engine's per-term Clifford work is the entire gap (11.8 s of a 12.0 s round set vs the frame's 7 ms). One physical constraint the experiment surfaced: the logical traffic must be *code-preserving* (diagonal + CNOT-type; no bare H on data), else the syndrome round measures the accumulated magic away — true of the toy and of the real protocols it models. Context that sharpens the niche: qiskit-aer `extended_stabilizer` *rejects* dynamic circuits outright (`mark`/`jump` unsupported), so online adaptive CH-form simulation is territory the flagship open-source implementation does not serve at all.

5.6 Episodic execution, measured

The episodic strategy (§3.3) is measured on feedforward conveyors against `clifft`'s exact record probabilities (`bench_episodic.py`).

Table 5: Episodic accuracy vs budget (conveyor, 6 episodes of 8 T's; exact per-episode rank 2^8=256; 24 repetitions; exact $P(\text{record}) = 1.15 \times 10^{-14}$).

budget k	naive rel-rms	naive bias	cross rel-rms	cross bias
32	2.86	+1.70	1.14	−0.004
64	1.38	+0.71	0.51	−0.199
128	0.57	+0.10	0.41	+0.047
256	0.40	+0.07	0.25	−0.024

The budget sweep (Table 5) shows the cross estimator doing its job: at k=32 the naive estimator carries a +1.70 bias where the cross estimator is unbiased to −0.004. Headline results. *Episodic-exact*: at budget $2^{m_{\text{blk}}}$ the run is exact ($\chi_{\text{peak}} = 2^{m_{\text{blk}}}$, P matching `clifft` to machine precision) — cost exponent m_{blk} instead of t_{live}, deterministically. *Scale*: a 72-qubit conveyor ($t_{\text{live}} = 72$;

whole-circuit χ would be 2^{72}) runs end to end with no 2^n object at $\chi \leq 400$, returning the record probability within 41% relative at 10× per-episode compression (mean of four cross estimates, 66 s). To be clear about who wins *this* instance: `clifft` compiles it to an m_{blk}-sized block (k=12) and answers exactly in ∼1 ms — the run demonstrates the mechanism at scale; the episodic strategy's own regime is conveyors whose *per-episode* block already breaks memory (the width-128, k=49 case of §1, where it is the only feasible strategy). *The measured limit*: for full-record probabilities the error turns on sharply below the exact budget and compounds steeply with the episode count (rel-rms 16 at R=8, 110 at R=16 even at 2× compression) — the dispatcher's R^2 cost model is optimistic there; episodic-exact or mild compression is the prescription for record targets, and the dispatcher prices both.

Is a 10^{-22} probability meaningful? Yes — it is *typical*, not pathological: a 72-bit record has probability $\sim 2^{-72} \approx 2 \times 10^{-22}$ if outcomes were uniform, so this is the natural size of *any* full-record probability at this width. Relative accuracy is therefore the right metric, and constant-factor accuracy of such quantities is exactly what likelihood workflows consume (cross-entropy benchmarking scores, per-record likelihood ratios, decoder likelihoods; a 41% relative error is 0.34 nats of log-likelihood, averaging out over records since the estimator is unbiased). No sampling method can access these quantities at all ($\sim 10^{22}$ shots for one expected hit — the measured 0/96 aer result of §5.7 is the same phenomenon at n=24). Conversely, applications needing only marginals or expectation values face polynomially-sized targets, where the record-probability breakdown regime is not the binding constraint.

5.7 External tools

On the same dense-IQP instances and targets as §5.1 (`bench_external.py`, everything validated against MitM). Table 6 consolidates the cross-method run times; the tools in detail:

Table 6: Cross-method run times: seconds for 96 record probabilities on shared dense-IQP instances (fastest per size in bold; note the guarantees differ).

method	guarantee	n=24	n=48	n=56	n=72
`clifft record_probabilities`	exact	**0.003**	17.3	398	∼1 TB, infeasible
MitM evaluator (ours)	exact	0.4	**4.7**	**84.5**	∼1 TB, infeasible
QuiZX	exact	0.5	251	2188	—
Quokka# + GPMC	exact	≈300	timeout	timeout	—
aer `extended_stabilizer`	sampling	455 (0/96 hit)	—	—	—
CH backend, δ=0.15	TV ≈ 0.08	0.08	16.9	86.2	—
CH backend, δ=0.4	TV ≈ 0.18	0.09	2.4	12.1	**267***

*single-shot build + one amplitude in 1.9 GB (Table 3) — the only method that runs at this size.

- **QuiZX** [9] (v0.3.0) computes each probability exactly, one ZX-simplify+decompose run per target; its per-amplitude cost grows at the exact-rank rate $2^{0.396n}$ (measured term counts $301 \to 1.9 \times 10^6$ over n=24 → 56: ZX simplification does not compress dense T+CZ IQP). For 96 targets: 0.5/251/2188 s at n=24/48/56 — vs this CH backend's 0.3/17/89 s at TV ≈ 0.08, and vs MitM's 0.4/4.7/84.5 s *exactly*. Two qualifiers: QuiZX answers a harder (exact) question, and on pure T+CZ IQP the structure-specific MitM ties the approximate backend

through $n \approx 56$ — the CH backend's edge over *all* exact methods opens past the $2^{n/2}$ wall or off the IQP structure.

- **Quokka#** [11] (with its GPMC model counter, built locally): exact per-amplitude, 3.1 s at n=24 (correct to 5×10^{-15}) but timeout (> 300 s/amplitude) already at n=32 — the dense CZ graph defeats the CNF encoding far below the tool's published sparse-circuit range. External exact ranking on this family: QuiZX ≫ Quokka#, with MitM fastest of all three.

- **qiskit-aer** `extended_stabilizer` (`bench_aer.py`; aer 0.17.2, settings inline): a sampling-only tool. On sampling, `clifft` sustains 1.8–4.4 × 10^6 exact shots/s (peak_rank = 1 families) vs aer's approximate 2721 → 44 shots/s over $n = 10 \to 28$ — ratios 1630× to 40,000×. On strong simulation it cannot enter: estimating 96 targets with true $P \sim 5.6 \times 10^{-8}$ at n=24, 30,000 shots (455 s) hit 0/96; `clifft` answers exactly in 3.5 ms. The three-way split: `clifft` owns exact work while $2^{n/2}$ fits; this backend owns approximate $P(x)$ beyond; `extended_stabilizer` (sampling-only, maintenance-frozen upstream) is dominated on both tasks.

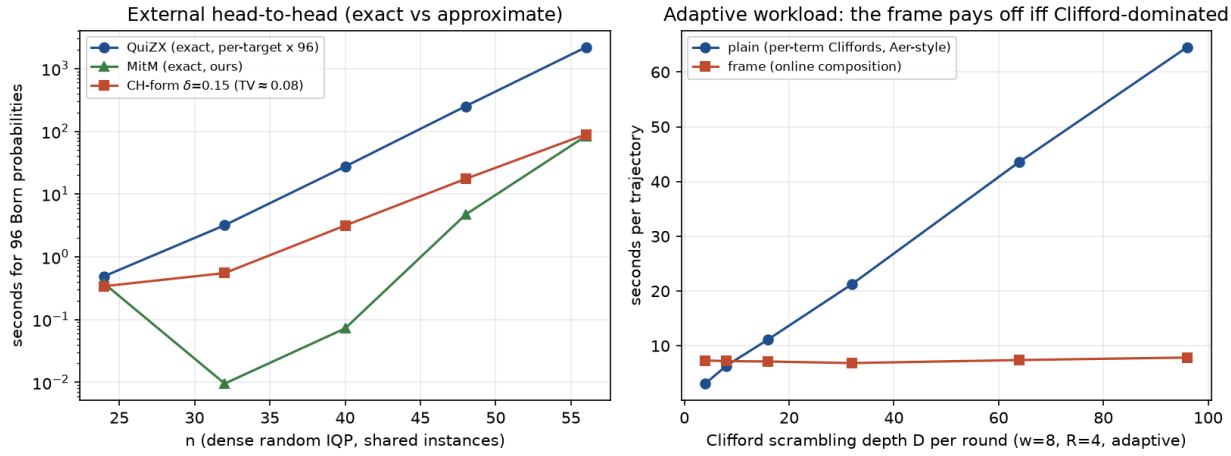


Figure 3: *Left:* external head-to-head on shared dense-IQP instances: QuiZX (exact, per-target) vs the MitM evaluator (exact) vs this CH backend (approximate, TV ≈ 0.08), seconds for 96 probabilities. *Right:* the adaptive-workload phase boundary at w=8: plain (per-term Cliffords) grows linearly in Clifford depth D; the frame composition is flat; crossover at $D^* \approx 1.2w$.

6 Limitations

- **Approximate**, with the guarantee of §3.5 (additive/typical, not per-x relative); the dense engine remains the tool wherever exactness is required and feasible.

- **The exact competition is $2^{n/2}$, not 2^n**, on the benchmark family; the CH backend's genuinely-unique regime starts near $n \approx 62$, and its speed advantage at moderate accuracy near $n \approx 34$–48 — always in terms of the *compiled peak rank*, which for most circuits is far below the qubit count.

- **Norm estimation** is the accuracy-cost bottleneck for generic non-product circuits that do not end fully collapsed ($\chi L \sim 1/\delta^4$); the clean results lean on the analytic and rank-1 normalization regimes.
- **Episodic compression of full-record probabilities** degrades sharply below the per-episode exact budget (§5.6); episodic-exact carries no such caveat.
- **The frame's advantage is constant/poly, not an exponent**, with substantial static prior art [2, 3, 6, 5]; its measured value is confined to Clifford-dominated adaptive circuits (Table 4).
- **Scale-arm gaps:** the C++ pipeline is build-and-query only (no measurement path — the gadgetized route removes the need for one in strong simulation) and consumes plain gate streams rather than HIR; the Python engine's sampling measurement path and canonical dedup materialize 2^n (validation-scale only; the forced-replay and episodic paths are non-materializing). The adaptive wall-clock numbers are Python-vs-Python (fair internally, absolute constants not comparable to C++ tools).
- `quEStab` (ACM ICS 2026, multi-GPU extended stabilizer) still needs a manual literature check (paywalled, no preprint).

7 Conclusion

The system is one compiler with two executors. `clifft`'s compiler reduces every circuit to a Clifford-free Heisenberg IR and, in doing so, computes the very quantities (k, t_{live}, m_{blk}, R) that price four execution strategies before anything runs. The dense engine keeps its exactness contract and its dominance wherever the compiled block is small — which, by the compiler's own strength, is most circuits. The CH backend extends reach where the block is large but magic is sparse: measured crossovers at $n \approx 34$–48 against the true exact fast path, sole feasibility past $n \approx 62$, a predictable accuracy dial (TV $= 0.50\,\delta$), generality over arbitrary Clifford+T circuits via gadgetization, adaptive circuits via the online frame (winning precisely on the fault-tolerance-shaped workloads it targets), and episode-structured computations at the per-episode exponent. Each claim carries its measured boundary: where `clifft` wins, where the estimator degrades, and where the only honest answer is "infeasible".

References

[1] S. Bravyi, D. Browne, P. Calpin, E. Campbell, D. Gosset, M. Howard, *Simulation of quantum circuits by low-rank stabilizer decompositions*, Quantum **3**, 181 (2019), arXiv:1808.00128.

[2] S. Bravyi, D. Gosset, *Improved classical simulation of quantum circuits dominated by Clifford gates*, Phys. Rev. Lett. **116**, 250501 (2016), arXiv:1601.07601.

[3] H. Qassim, J. J. Wallman, D. Gosset, *Clifford recompilation for faster classical simulation of quantum circuits*, Quantum **3**, 170 (2019), arXiv:1902.02359.

[4] H. Qassim, H. Pashayan, D. Gosset, *Improved upper bounds on the stabilizer rank of magic states*, Quantum **5**, 606 (2021), arXiv:2106.07740.

[5] H. Pashayan, O. Reardon-Smith, K. Korzekwa, S. D. Bartlett, *Fast estimation of outcome probabilities for quantum circuits*, PRX Quantum **3**, 020361 (2022), arXiv:2101.12223.

[6] H. J. García, I. L. Markov, *Simulation of quantum circuits via stabilizer frames*, IEEE Trans. Computers **64**, 2323 (2015), arXiv:1712.03554.

[7] F. C. R. Peres, E. F. Galvão, *Quantum circuit compilation and hybrid computation using Pauli-based computation*, Quantum **7**, 1126 (2023), arXiv:2203.01789.

[8] A. C. Nakhl et al., *Stabilizer tensor networks with magic state injection*, Phys. Rev. Lett. **134**, 190602 (2025), arXiv:2411.12482.

[9] A. Kissinger, J. van de Wetering, *Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions*, Quantum Sci. Technol. **7**, 044001 (2022), arXiv:2109.01076.

[10] M. J. Bremner, A. Montanaro, D. J. Shepherd, *Average-case complexity versus approximate simulation of commuting quantum computations*, Phys. Rev. Lett. **117**, 080501 (2016), arXiv:1504.07999.

[11] J. Mei, M. Bonsangue, A. Laarman, *Simulating quantum circuits by model counting*, CAV 2024, arXiv:2403.07197 (Quokka#).